AI Engineering and Claude Architect · Part V, Claude Code

Claude Code, Lesson 3

Commands, Skills, Subagents and Hooks

A safety hook crashed on odd input, exited with code 1, and the blocked action went ahead. Hooks are real enforcement, and a hook that fails open isn't. Four extension points, four different jobs.

Module	AI12 of 13
Lesson	3 of 5
Prerequisites	AI12 L1–L2, AI06 L6 and L8, AI09 L3
Exam relevance	**Core: TS 3.2.** Also TS 1.3, 1.5, 2.3, 2.4
Reading time	~40 minutes

Contents

1. Commands, Skills, Subagents and Hooks

Lesson 3 of 5 · Claude Code

Commands, Skills, Subagents and Hooks

Key idea

Choose by when something should load and who enforces it. CLAUDE.md is always loaded. Skills load on demand. Subagents isolate work in their own context. Hooks are code that the harness runs regardless of what the model decides. Only hooks and deny rules are guarantees, and a hook must fail closed to be one.

> *Before: hierarchy and scope, multi-agent basics, hooks in the SDK. Teaches: commands and skills with frontmatter, `context: fork`, subagent definitions and tool limits, hook events and exit codes, and MCP server scope. Used by: L4, L5, AI13.*

The hook: a guard that fails open

I wrote a `PreToolUse` hook that protects `.env` and generated code, and ran it as a real process against six events shaped from the docs. **The script ran for real, though not inside a live Claude Code session, as that would have spent model tokens.** Real output from `HookLab.java`:

```
event                       naive hook                      safe hook
Edit src/App.java           proceeds                        proceeds
Edit .env                   BLOCKED                         BLOCKED
Write src/gen/Client.java   BLOCKED                         BLOCKED
Bash: cat .env              proceeds                        BLOCKED
Bash: cat .e*               proceeds                        proceeds
malformed input             proceeds (hook crashed, exit 1) BLOCKED
```

Three lessons sit in those rows:

1. **A crash isn't a block.** The docs say exit 2 blocks, and that with other exit codes the action proceeds on most events. An uncaught exception exits 1. **An unhandled error in a safety hook lets the**

action through, so catch everything, and exit 2.

2. **Other tools reach the same file.** A hook matching only `Edit|Write` never sees `cat .env` through Bash.
3. **Text checks have gaps.** `cat .e*` passes. The docs recommend the sandbox for filesystem and network enforcement that doesn't depend on a command's text (AI09 L3).

Intuition: staff in a hospital

Consider a hospital's standing rules on every ward's wall. That's CLAUDE.md. A procedure binder fetched for one operation is a skill. A specialist called in to report back is a subagent. And the drug cabinet that won't open without a scan is a hook.

You wouldn't laminate every procedure on the wall. Nor would you trust a wall poster to keep the cabinet locked.

Mechanism

Which one?

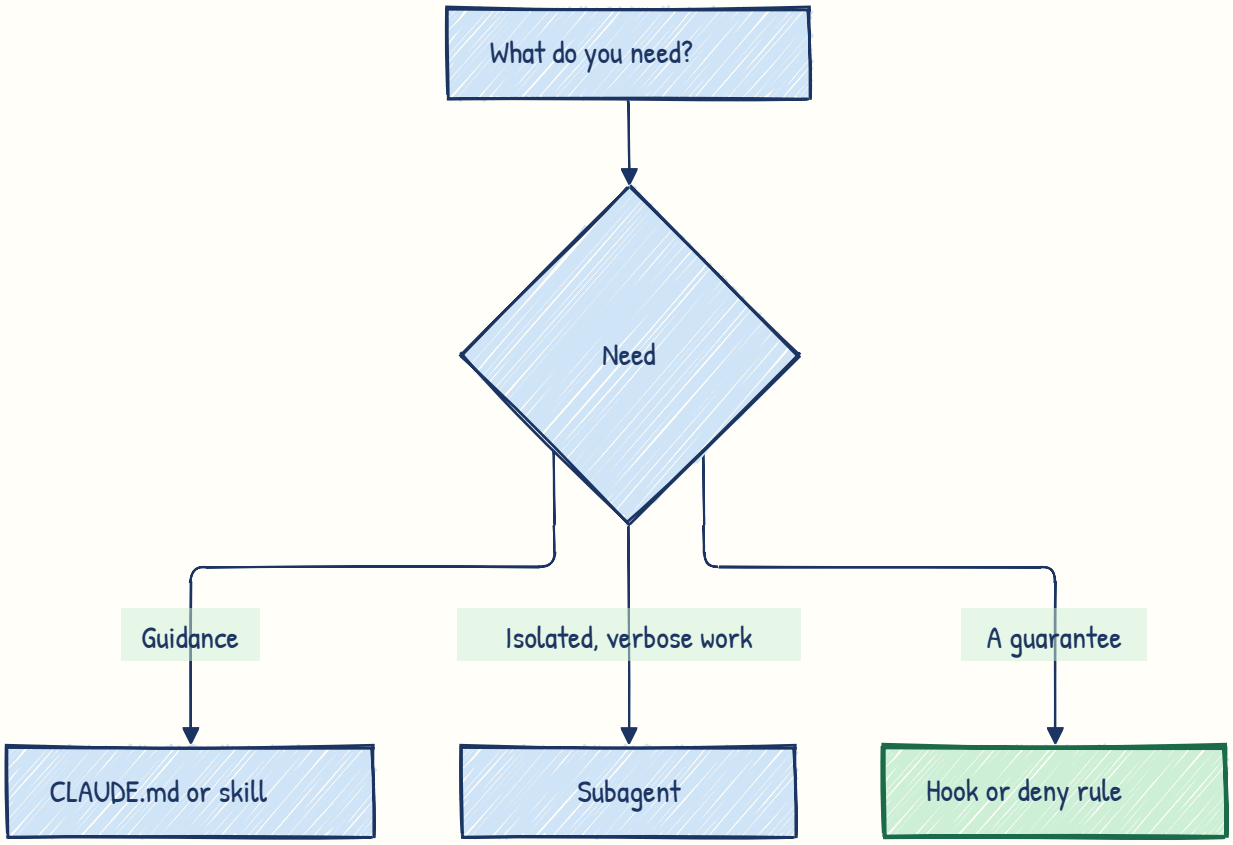


Figure 1. Ask what you need: always-on context, an on-demand procedure, isolation, or a guarantee.

Mechanism	Loads	Enforced?	Use for
CLAUDE.md	Always	No	Universal standards
Rule with `paths`	On matching files	No	File-type conventions
Skill or command	When invoked or relevant	No	Task-specific procedures
Subagent	When delegated	Its tool limits are	Isolated work, with a summary back
Hook	On events	**Yes**	Must-always, must-never
MCP server	Tools at connection	No	External systems

Commands and skills

Location	Scope
`.claude/commands/x.md`	Project, shared through Git
`~/.claude/commands/x.md`	Personal
`.claude/skills/x/SKILL.md`	Project, shared through Git
`~/.claude/skills/x/SKILL.md`	Personal

Exam answer and what to build today

The guide treats **commands and skills** as separate, and the exam will likely use that framing. The current docs merge them: `.claude/commands/deploy.md` and `.claude/skills/deploy/SKILL.md` both create `/deploy`, and if both exist, the skill wins. Scope order on name clashes is enterprise, then personal, then project. So to make a variant safely, the guide's advice stands: use a **different name** in `~/.claude/skills/`.

Frontmatter. The exam guide names three fields, and the current docs list more:

Field	Does	On the exam?
`context: fork`	Runs the skill in a subagent, with no conversation history	**Yes**
`allowed-tools`	See the note below	**Yes**
`argument-hint`	Hints for the expected arguments	**Yes**
`description`	When to use it, and how Claude decides to load it	Implied
`disable-model-invocation`	Only you can start it	No
`user-invocable: false`	Only Claude can start it	No
`agent`	Which subagent type runs a forked skill	No
`paths`	Load only for matching files	No
`model`, `effort`	Override for that turn	No

Exam answer and what to build today

The guide says `allowed-tools` **restricts** tool access during a skill, and an exam item may use that. The docs say it's "Tools Claude can use without asking permission during the turn that invokes this skill," and that it doesn't restrict other tools. To really block tools, build with deny rules. In a subagent, its `tools` list does restrict.

`context: fork` **in practice.** It suits verbose output such as codebase analysis, and exploration like brainstorming, which would otherwise crowd or influence later work.

```
---
description: Analyze module structure and report dependencies
context: fork
agent: Explore
argument-hint: "[module-path]"
---
Map the dependencies of $ARGUMENTS. Report entry points with file paths.
```

`$ARGUMENTS` holds the argument string. The docs also describe `` !`command` ``, which runs a shell command before the skill's content is sent, and puts its output in place.

Context cost. By default, a skill's description is always in context, and the body loads on use. So many skills still cost something. The docs advise keeping `SKILL.md` under 500 lines, with supporting files alongside it.

Subagents

AI06 L6 taught the pattern. In Claude Code, from the subagents page:

Fact	Detail
Definitions	`.claude/agents/` for a project, `~/.claude/agents/` for you
Required fields	`name` and `description`
Tools	`tools` allowlist, and `disallowedTools` denylist
Context	Fresh window. It doesn't see your conversation, and only its summary returns
Built-ins	Explore and Plan (read-only), and general-purpose
Spawning tool	**Agent**, renamed from Task in v2.1.63, with `Task(...)` kept as an alias
Nesting	Can spawn up to three layers below the main conversation, by default
Unknown fields	Ignored without an error

```
---
name: safe-researcher
description: Research agent with restricted capabilities
tools: Read, Grep, Glob, Bash
---
```

The docs' guidance for choosing: stay in the main conversation for back-and-forth and quick changes, and use a subagent when work produces verbose output, needs enforced tool limits, or can return a summary.

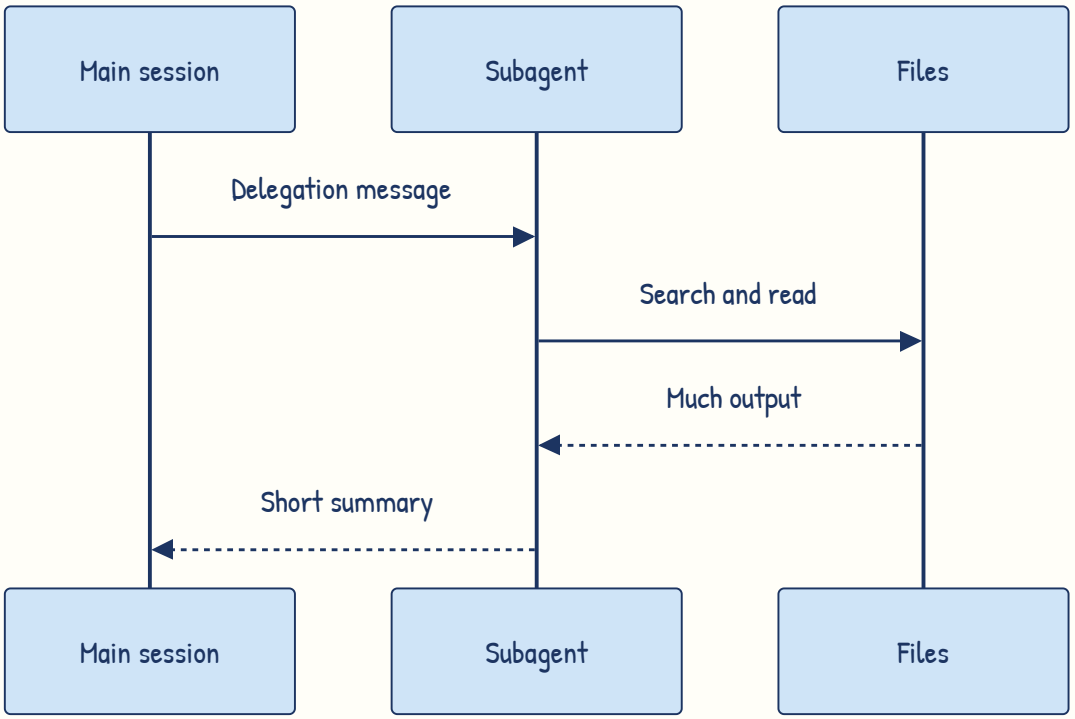


Figure 2. The subagent's working stays in its own context. The main conversation receives a summary, and that's why verbose work belongs there.

Hooks

AI06 L8 covered them for the SDK. In Claude Code, you configure them in settings files. Some events, from the guide:

Event	When	Common use
`SessionStart`	Session begins or resumes	Load context
`UserPromptSubmit`	Before a prompt is processed	Add or block
`PreToolUse`	Before a tool runs. **Can block**	Protect files, enforce rules
`PermissionRequest`	When a decision is needed	Auto-approve narrowly
`PostToolUse`	After a tool succeeds	Format code, normalize results
`SubagentStop`	When a subagent finishes	Check its work
`Stop`	When Claude finishes responding	Require passing tests

Exit codes, from the docs:

Exit	Meaning
0	Proceed. Stdout can carry JSON for finer control
2	Block. Stderr becomes feedback, on events that can be blocked
Other	The action proceeds on most events, with a note

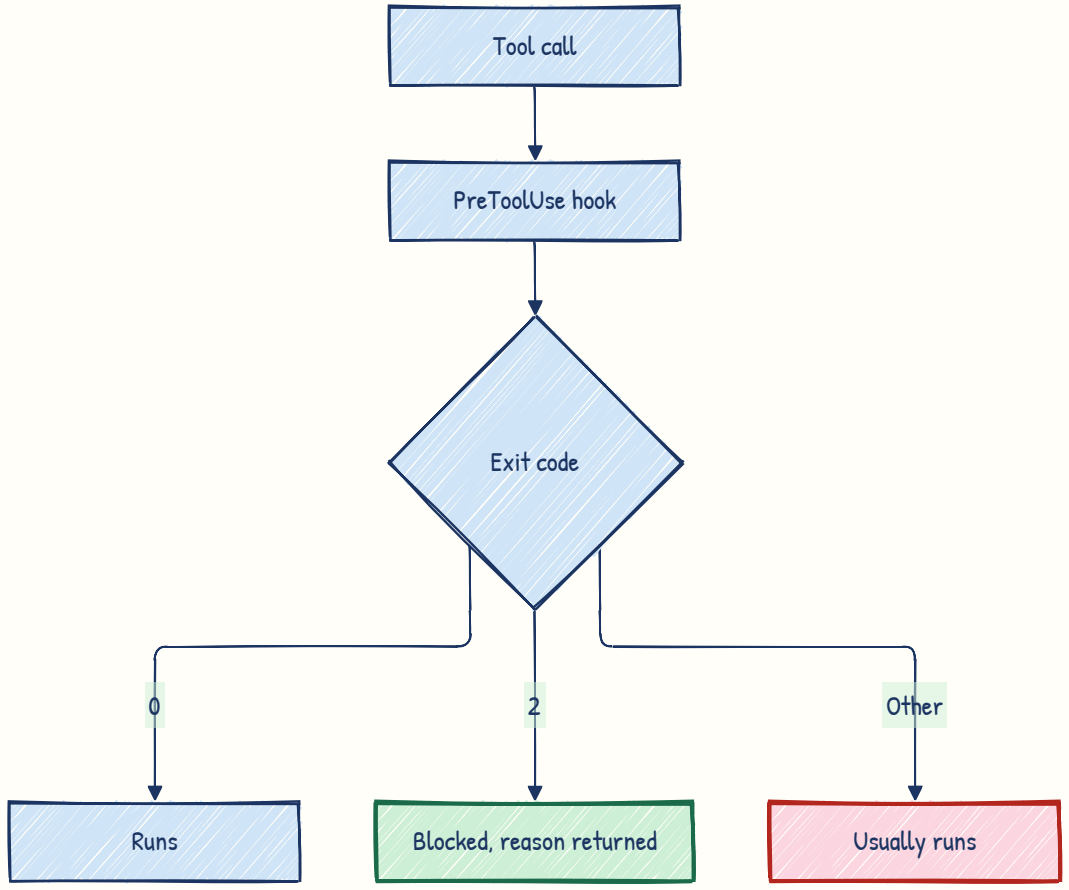


Figure 3. Only exit 2 blocks. A crash is usually a non-2 exit, so it fails open unless you catch it.

A settings example, for shape:

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{ "type": "command", "command": "java -cp out course.cc.ProtectFiles safe" }]
    }]
  }
}
```

Limits and precedence, from the docs:

Fact	Meaning
`PostToolUse` can't undo	The tool has already run
A hook "allow" doesn't bypass deny or ask rules	Deny-first still holds
A blocking hook beats allow rules	Exit 2 stops it first
`Stop` fires on every finish	Not only on task completion
Other hook types exist	`http`, `prompt`, and `agent`, for judgment using a model
`-p` in an untrusted folder	Project hooks and `.mcp.json` servers still run, with no trust dialog (L5)

A hook's speed matters. It runs on every matching call. A JVM start measured a median of 150–180 ms on this laptop, across my runs. That's fine on edits, and may add up if it matches every tool.

MCP in Claude Code

TS 2.4, from AI06 L4. Quick recap:

Item	Where
Shared servers	`.mcp.json` in the project
Personal or experimental	`~/.claude.json`
Secrets	`${VAR}` expansion, set per developer

Prefer existing servers for standard integrations. Tools from all servers are available together, so more servers make selection harder (TS 2.3).

Why the obvious fixes fall short

Obvious idea	Why it falls short
Put procedures in CLAUDE.md	Loaded on every task, at a cost
Put must-never rules in a skill	Skills are guidance
Use `allowed-tools` as a block	Docs: it pre-approves, and doesn't restrict
Match only Edit and Write	Bash can reach the same file
Trust a hook without tests	A crash can fail open
Run everything in subagents	Slower for quick, iterative work

How Anthropic solved it

Mechanism	What's published	Source
Skills	Scope, frontmatter, dynamic context	Skills docs
Subagents	Definitions, tool limits, isolation	Subagents docs
Hooks	Events, exit codes, JSON, types	Hooks guide and reference
Enforcement model	Deterministic or non-deterministic	Best practices
MCP	Scopes and expansion	MCP docs

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: long prompts retyped by each developer.

Improved: shared skills. New problem: skills are advisory, and "always" rules still fail sometimes.

Production: each mechanism does what it's good at, and it's all versioned.

Component	Why it exists	How it communicates
Skills in Git	Shared procedures	Invoked by name
Forked analysis skills	Keep verbose output out	Subagent summary
Role subagents	Enforced tool limits	Agent tool
PreToolUse hooks	Must-never rules	Exit codes
PostToolUse hooks	Formatting and normalization	After each edit
Stop hook or CI	Must pass before "done"	Tests
Sandbox and deny rules	Limits independent of text	Settings
Managed settings	Rules developers can't override	Admin deployment

Failures:

Failure	How you'd notice	What to do
Hook crash	Protected file changed	Catch all errors, exit 2, and test it
Hook too slow	Sessions drag	Narrow its matcher, and use a faster runtime
Skill never triggers	Procedure isn't followed	Improve `description`, or invoke by name
Subagent has too much power	Unexpected edits	`tools` allowlist, and check field names
Typo in frontmatter	Silent	Lint (L2)
Too many skills	Descriptions fill context	Prune, or `disable-model-invocation`

Java

The safe hook's shape is the point: every path out is a decision.

```
System.exit(2);
```

A real block is exit 2, and the naive version shows the alternative. It rethrows, so the JVM exits with 1, and on most events the action proceeds:

```
throw e;
```

Hooks read JSON on stdin, so any language works. Java's start-up cost is the main reason people choose a lighter runtime for frequent hooks.

Trade-offs

Decision	Benefit	Cost
Skill	Loads when needed	May not trigger
CLAUDE.md	Always there	Attention on every task
Subagent	Isolation, and tool limits	Loses conversation context
Fork a skill	Clean main session	Can't refer to the conversation
Hook	Guarantee	Code to maintain, and latency
Hook on preference	Rigid	Docs say to delete these

When not to use a hook: preferences that have sensible exceptions. Current models judge those well from the code in front of them.

Common mistakes

Writing a security hook that fails open. Treat any unexpected condition as a block, and test with malformed input.

Common mistakes

Misspelling a subagent field. `allowedTools` is ignored without an error, so the agent keeps every tool. The field is `tools`.

What to say in an interview

"I choose by loading and enforcement. CLAUDE.md is always loaded, skills are on demand, and subagents isolate work. Only hooks and deny rules guarantee anything. A hook blocks only on exit 2, so it has to catch errors and fail closed, and it mustn't rely on matching only Edit when Bash reaches the same file. I also know that the guide says `allowed-tools` restricts, while current docs say it pre-approves."

Certification lens

How this lesson maps to CCAR-F

TS 3.2 is exam content: project and user scope, `context: fork`, `allowed-tools`, `argument-hint`, personal variants with different names, and skills versus CLAUDE.md. It supports TS 1.3 (Task, now Agent), TS 1.5 (hooks) and TS 2.4 (MCP scope).

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Code Generation · Medium. Your `/dependency-map` skill prints long output that crowds the session. What's the best fix?

A. Set `argument-hint` to limit its output

B. Add `context: fork` to its frontmatter

C. Move its instructions into `CLAUDE.md`

D. Ask developers to run `/clear` afterward

A B (TS 3.2). A fork isolates verbose output. A only hints at arguments, C loads it on every task, and D loses the session's context.

Q Q2 · Exam style, scenario: Code Generation · Easy. Which belongs in a skill rather than `CLAUDE.md`?

A. A release checklist that's used before each release

B. The project's test command

C. The rule that money uses `BigDecimal`

D. The location of shared error helpers

A **A** (TS 3.2). It's task-specific and occasional. The other three apply across all work.

Q Q3 · Exam style, scenario: Developer Productivity · Medium. Generated code in `src/gen/` must never be edited by the agent. Which is the right control?

A. A CLAUDE.md line in capital letters

B. A skill that explains generated code

C. A rule with `paths: ["src/gen/**"]`

D. A `PreToolUse` hook that blocks edits there

A **D** (TS 1.5). "Never" needs enforcement. The other three are guidance, however they're scoped.

Q Q4 · Exam style, scenario: Code Generation · Medium. You want your own variation of the team's `/review` skill, with no effect on anyone else. What should you do?

A. Edit `.claude/skills/review/SKILL.md`, and don't commit it

B. Create a differently named skill in `~/.claude/skills/`

C. Add your changes to the team's skill behind a comment

D. Create another skill named `review` in `.claude/commands/`

A **B** (TS 3.2). A personal, differently named skill avoids any effect. A is fragile, C changes the shared file, and D conflicts with it.

Q Q5 · A safety hook throws a null pointer exception on an unusual tool input. What happens, and how should it be written?

A It exits non-zero and not 2, so on most events the action proceeds, and that means the guard fails open. Catch every error, exit 2 with a reason, and test malformed input. Consider sandboxing as well, since hooks that read text have gaps.

Revision - Lesson 3

→ Choose by loading and enforcement: CLAUDE.md, skill, subagent, hook.

→ Commands and skills are project or user scoped. Current docs merge them.

→ `context: fork` isolates a skill. `argument-hint` prompts for arguments.

→ Guide: `allowed-tools` restricts. Docs: it pre-approves that turn. Use deny rules to block.

→ Personal variants go in `~/.claude/skills/`, with different names.

→ Subagents: fresh context, summary back, `tools` allowlist. Unknown fields are silently ignored.

→ Task is now Agent. `Task(...)` still works as an alias.

→ Only exit 2 blocks. A crash usually fails open, so catch errors.

→ A hook on Edit doesn't see Bash, and text checks have gaps. Use sandbox limits as well.

→ `.mcp.json` for team, `~/.claude.json` for you, `${VAR}` for secrets.

Your notes